

# Life Orchestrator — Resume Bullets by Role Type

**Author:** Ismail Abdullah (Ismailab1)

**Copyright:** © 2026 Ismail Abdullah (Ismailab1). All rights reserved.

**Purpose:** Ready-to-use, truthful resume bullet points drawn directly from verifiable project evidence. Each section is tailored to a specific role type. Select the section(s) that match your target job and mix/match bullets as needed.

**Honesty policy:** Every bullet below corresponds to something actually built in this repository. Do not add claims not supported here.

---

## Role: Software Engineer / Full-Stack Engineer

- Built a full-stack AI-powered SPA in **React 19 + TypeScript** with a minimal **Express.js** backend proxy, deployed on **Google Cloud Run** via multi-stage Docker containers.
  - Architected a **local-first data model** using browser `localStorage` with a proactive quota management system — real-time usage display, amber/red threshold warnings, granular date-level cleanup, and a Base64-aware storage formula.
  - Designed and implemented a **TypeScript type system** ( `types.ts` ) covering the full application domain: tasks, relationship ledger, chat history, orchestration proposals, and AI memory — enabling compile-time safety across all data flows.
  - Implemented **streaming AI responses** with token-by-token UI rendering, `AbortController` -based cancellation, and a custom `withTimeout()` utility enforcing 30-second stream, chunk, and 60-second orchestration timeouts.
  - Engineered an **inline editing UX pattern** (click-to-edit, auto-save on blur) across all data-entry components, eliminating modal dialogs and reducing user interaction overhead.
  - Built a **view state machine** for client-side navigation in place of a full router, keeping navigation logic consolidated and reducing configuration surface area.
  - Implemented **background orchestration with debounce protection**: task additions confirm immediately; AI re-orchestration fires 3–6 seconds later via a separate async message, eliminating previous multi-minute blocking.
  - Wrote an **image compression pipeline** ( `imageService.ts` ) resizing uploads to 800×800 px at JPEG 70% quality, enforcing a 5-image/day upload limit to protect the 5 MB `localStorage` quota.
- 

## Role: AI / ML Engineer / AI Application Developer

- Integrated **Google Gemini 2.5 Pro** via the `@google/generative-ai` SDK, implementing function calling (tool use) as the primary AI-application interface.
  - Designed **7+ Gemini tool executor functions** with TypeScript-validated JSON schemas: `save_day_plan` , `update_relationship_status` , `log_checkin` , `save_memory` , `propose_orchestration` , `get_life_context` , `get_relationship_status` .
  - Built a **temporal mode system** where the AI operates in one of three behavioral profiles — Reflection (past dates), Active (today), Planning (future) — via dynamically composed system instructions, preventing scheduling logical impossibilities.
  - Implemented **defense-in-depth AI safety**: Gemini schema validation → TypeScript compile-time checks → runtime executor validation → `localStorage` isolation, ensuring the LLM cannot execute arbitrary code or access system resources.
  - Engineered a **persistent AI memory system** ( `Memory[]` in `localStorage` ) with FIFO enforcement at 100 items, enabling user preference learning and long-term context continuity across sessions.
  - Designed **prompt engineering as application logic**: the system prompt ( `SYSTEM_INSTRUCTION` ) encodes all orchestration rules, priority balancing, and communication tone — making AI behavior testable and iterable without code changes.
  - Implemented **streaming UX** for LLM responses using `generateContentStream` with chunk-level timeout monitoring and intelligent fallback message generation when the primary stream stalls.
  - Built a **session isolation system**: AI context resets automatically on date navigation, preventing context bleed between dates and ensuring each session operates on the correct temporal data.
-

## Role: Frontend / UI Engineer

- Built a **responsive SPA** in React 19 + TypeScript using Tailwind CSS utility classes — zero hand-written CSS, aggressive class reuse, ~5 KB purged CSS output in production.
  - Implemented **real-time token-by-token AI text streaming** in the chat UI, with `isThinking` state flags, streaming cursors, and graceful error/timeout states.
  - Designed a **multimodal chat interface** supporting text, Base64-encoded images, voice input (microphone permissions), embedded orchestration proposal cards, and contact proposal cards — all in a single chat stream.
  - Built **toast notification infrastructure** ( `Toast.tsx` ) providing real-time user feedback for all major actions, errors, and validations with auto-dismiss behavior.
  - Implemented an **optimistic UI pattern** across task and relationship editing: state updates immediately on user action, with async AI confirmation arriving separately — eliminating perceived latency.
  - Configured **Tailwind CSS** with `@tailwindcss/typography` plugin for `prose`-class AI markdown rendering, content-path scanning for production purging, and responsive design utilities.
  - Built a **storage management UI** ( `StorageManager.tsx` ) with a visual quota percentage bar, per-data-type breakdown table, and date-granular message deletion controls.
  - Integrated **Google Calendar** sync UI with OAuth popup flow, requiring custom `Cross-Origin-Opener-Policy` headers configured in both the Express server and Vite dev server.
- 

## Role: Backend / Platform / DevOps / Cloud Engineer

- Designed and deployed a **multi-stage Docker build** (Node 22 Alpine): Stage 1 compiles the React/Vite frontend; Stage 2 serves it alongside a minimal Express.js API proxy — production image excludes all devDependencies and build tools.
  - Deployed to **Google Cloud Run** (serverless containers): auto-scaling to zero, environment-variable-based secret injection, no VM management overhead.
  - Built a **secure API proxy layer** in Express.js that intercepts client Gemini requests, strips placeholder keys, injects the real `GEMINI_API_KEY` server-side, and forwards requests with streaming preserved — API key never exposed in client bundles.
  - Configured **CORS and security headers** ( `Cross-Origin-Opener-Policy: same-origin-allow-popups` ) required for Google OAuth popup flow, applied both in Express middleware and the Vite dev-server configuration.
  - Authored a **PowerShell deployment automation script** ( `advanced_deploy.ps1` ) orchestrating local Docker build, image push, and Cloud Run service update in sequence.
  - Implemented **SPA routing fallback** ( `app.get('*')` ) in Express to serve `index.html` for all unknown routes, enabling client-side `react-router-dom` navigation to work correctly after hard reloads.
  - Managed **environment variable injection** at two levels: Vite build-time injection for development ( `define: { 'process.env.API_KEY': }` ) and Cloud Run runtime injection for production.
  - Configured `.gcloudignore` to exclude `node_modules` , `.git` , and source files from Cloud Build submission, optimizing transfer size and build performance.
- 

## Role: Integration / Automation / API Engineer

- Built a **Google Calendar API integration** ( `googleCalendarService.ts` ) using GAPI and Google Identity Services (GSI) libraries — OAuth 2.0 popup authentication, bi-directional event sync, and recurring event support via RRULE.
  - Designed a **typed API abstraction layer** mapping Google Calendar event schema ( `GoogleCalendarEvent` ) to internal task types, with attendee, conference data, and organizer metadata support.
  - Implemented **AI function-calling integration** as a structured API contract: TypeScript schemas define what the AI can and cannot do, acting as both an integration surface and a safety boundary.
  - Built a **session context management system** that validates, injects, and resets AI session state — ensuring each API session operates with the correct date context and preventing stale data from bleeding across sessions.
  - Integrated the **@google/generative-ai SDK** with custom streaming, abort, and timeout infrastructure — wrapping the SDK's raw capabilities with production-grade reliability layers.
-

## Role: Product-Minded Engineer / Generalist

- Designed and shipped an **end-to-end AI life management product**: from data model design and prompt engineering through containerized deployment — solo project ownership across the full stack.
- Identified the core **UX insight** that traditional productivity tools are form-heavy, and built a conversational-first interface where natural language replaces form fields — reducing cognitive overhead for all task creation.
- Invented a **Kinship Debt Algorithm** ( $\text{Priority} \times \text{Days Since Contact}$ ) to quantify relationship health mathematically, enabling the AI to proactively surface neglected relationships without subjective guesswork.
- Made deliberate **build-vs-buy decisions**: no Redux (props sufficient), no React Router (state machine sufficient), no form library (inline editing sufficient) — adding dependencies only when complexity is justified.
- Documented all **architectural trade-offs inline** (design decision comments throughout `types.ts`, `vite.config.ts`, `tailwind.config.js`) making the codebase self-documenting for future contributors.
- Built a **multi-phase roadmap** from current MVP through IndexedDB migration, vector embedding search, native audio (Gemini Live API), and a Recursive Summary Engine for decade-scale context management.

---

*All bullets are verifiable in the repository. Do not use in applications without confirming direct experience with each claim.*